

SIMATS ENGINEERING
CO2_ASSESSMENT TOOL3_ Dry Run Evaluation

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1709

Register Number	192412199
Name	P POORNA CHANDU

COURSE OUTCOME COVERED

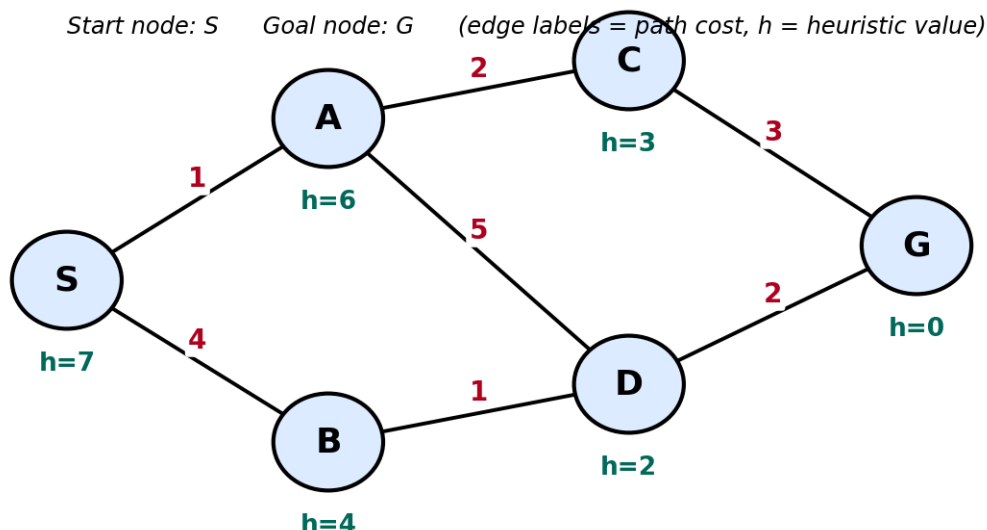
CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

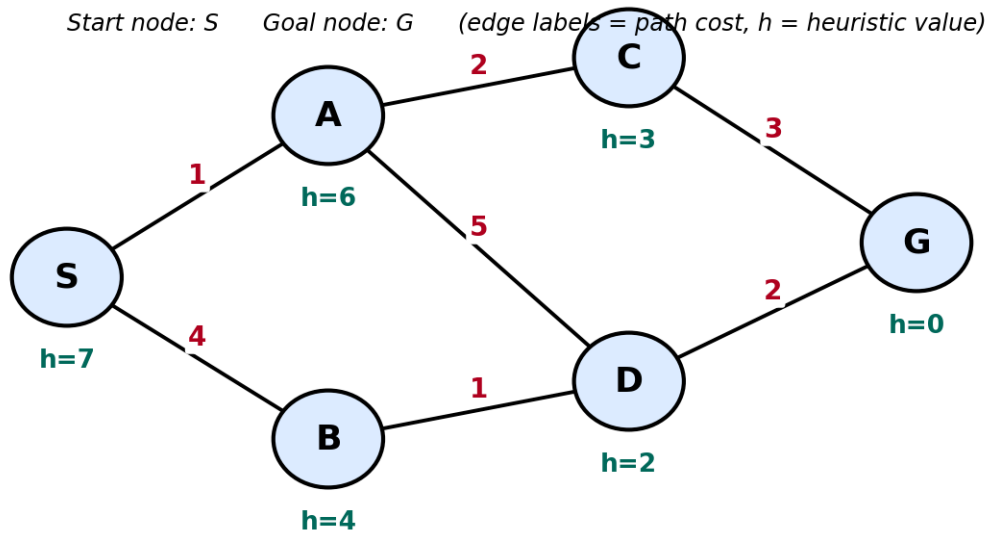
Total Marks:50

Q1. Consider a suitable state-space graph (with heuristic values $h(n)$ assigned to each node) of your choice for a given problem. Perform a dry run of Greedy Best-First Search from the start node to the goal node. Clearly show the frontier/open list, the node selected for expansion at each step, and the final path obtained.



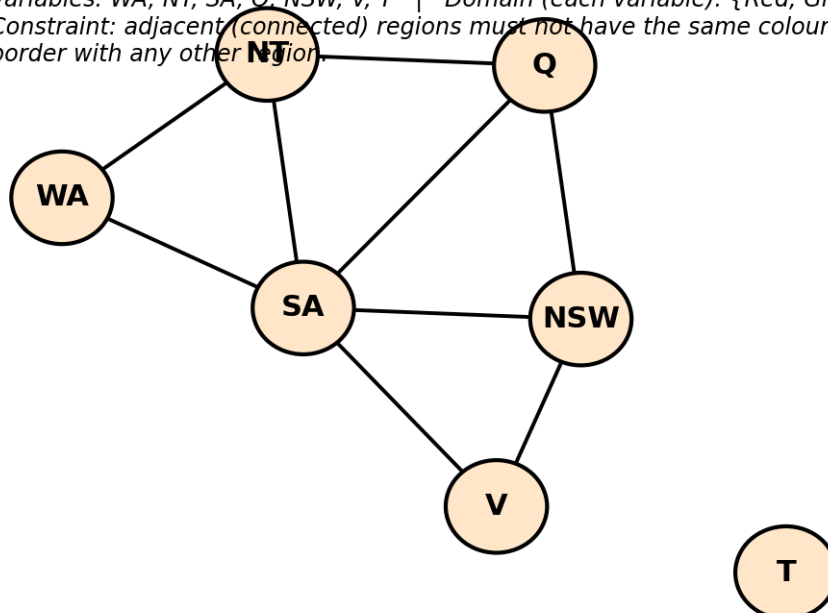
Q2. For a suitable weighted state-space graph (with edge costs and heuristic values $h(n)$) of your choice, perform a dry run of A* Search from the start node to the goal

node. Show the $g(n)$, $h(n)$ and $f(n)$ values computed at every step, the order in which nodes are expanded, and the final optimal path along with its total cost.



Q3. Formulate a given Constraint Satisfaction Problem (e.g., Map Colouring / N-Queens) by clearly stating the variables, their domains, and the constraints involved. Perform a dry run of the Backtracking Search algorithm, showing each variable assignment, every constraint check, and any backtracking steps, until a complete solution (or failure) is reached.

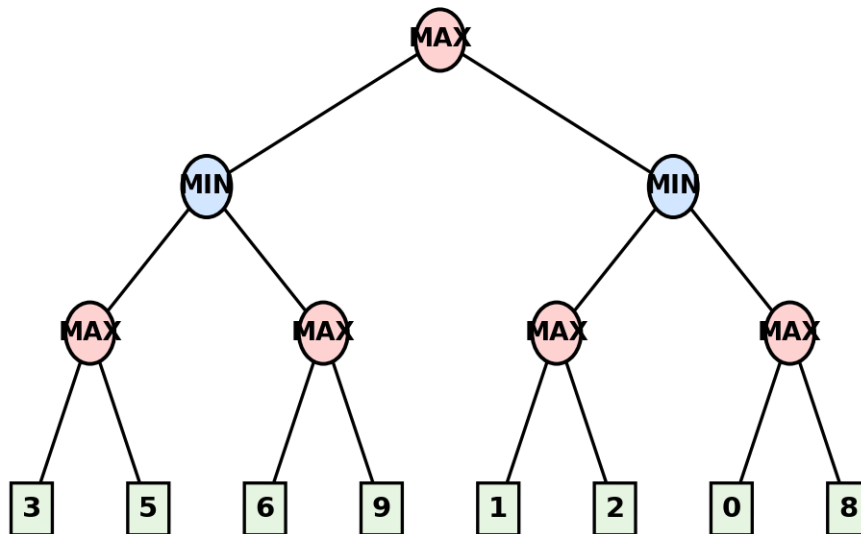
Variables: WA, NT, SA, Q, NSW, V, T | Domain (each variable): {Red, Green, Blue}
 Constraint: adjacent (connected) regions must not have the same colour. T has no shared border with any other region.



Q4. For a given two-player game tree of your choice (at least 3 levels deep), perform a dry run of the Minimax algorithm. Show the utility values at the leaf/terminal nodes

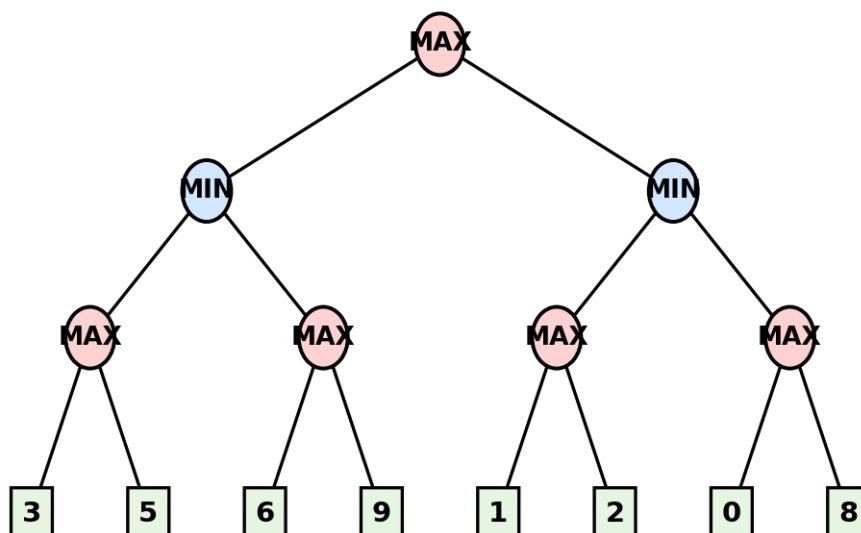
and the backed-up values computed at every internal node, and state the best move for the MAX player with justification.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Q5. Using the same (or a different) game tree from Q4, perform a dry run of the Minimax algorithm with Alpha-Beta Pruning. Show the α and β values maintained at each node during the traversal, clearly indicate which branches get pruned and why, and state the final best move obtained.

Root = MAX to move. Squares at the bottom are terminal (leaf) nodes with their utility values.



Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Dry Run Evaluation**RUBRICS FOR CALCULATION**

Criteria	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning	Max Marks
Problem Formulation & Setup (states/nodes, heuristic values or CSP variables, domains and constraints correctly identified)	Accurately identifies all required elements (states, heuristics/costs, or variables/domains/constraints) with clear, correct notation.	Identifies most required elements correctly, with only minor omissions or notational errors that do not affect the dry run.	Identifies some required elements; noticeable errors or missing components affect the later steps.	Little or no correct problem formulation; required elements are largely missing or incorrect.	10
Correct Application of Algorithm Procedure (expansion order, backtracking, or pruning as per the standard method)	Follows the exact algorithmic procedure at every step with no deviation from the standard method.	Follows the procedure correctly for most steps, with only one or two minor deviations that do not change the outcome.	Several steps deviate from the correct procedure, though the overall approach is recognisable.	Algorithm steps are largely incorrect, or the procedure is not followed.	10
Accuracy of Computations (g(n), h(n), f(n) values, minimax backed-up values, or α - β updates)	All computed values are accurate and consistent throughout the dry run.	Minor calculation errors are present but do not significantly affect the	Multiple calculation errors are present that affect the correctness of intermediate steps.	Calculations are mostly incorrect or missing.	10

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Max Marks
		final outcome.			
Correctness of Final Outcome (final path, CSP solution, or best move obtained)	Final result is completely correct and matches the expected optimal outcome.	Final result is mostly correct, with only a minor deviation from the optimal outcome.	Final result is only partially correct.	Final result is incorrect or not obtained.	10
Clarity of Presentation & Justification (trace tables/trees/diagrams, explanation of each step)	Dry run is presented clearly using well-organised tables/trees/diagrams, with every step explained and justified.	Dry run is reasonably clear and mostly organised, with minor gaps in explanation.	Dry run presentation is unclear or poorly organised, with limited justification.	Dry run is disorganised or illegible, with little to no justification.	10
				Total	50

Q1. Greedy Best-First Search

State-space graph used

Nodes: S (start), A, B, C, D, G (goal).

Edges (undirected, with path costs): S–A = 1, S–B = 4, A–C = 2, A–D = 5, B–D = 1, C–G = 3, D–G = 2.

Node	S	A	B	C	D	G
h(n)	5	4	2	2	1	0

Greedy Best-First Search expands the node with the lowest heuristic value h(n) only (edge costs are ignored while choosing).

Dry run

Step	Node expanded	Frontier before expansion (sorted by h)	Children generated
1	S	S(5)	A(4), B(2)
2	B	B(2), A(4)	D(1)
3	D	D(1), A(4)	G(0)
4	G (goal — stop)	G(0), A(4)	—

Path obtained: S → B → D → G, with actual path cost 4 + 1 + 2 = 7.

Note: this path is not the cheapest one available (S–A–C–G costs $1 + 2 + 3 = 6$). This is expected — Greedy Best-First Search is not optimal because it ignores $g(n)$ and is guided purely by $h(n)$.

Q2. A* Search

Same graph and heuristic table as Q1 (the heuristic is admissible and consistent for this graph).

Dry run — $g(n)$, $h(n)$, $f(n) = g(n) + h(n)$

Step	Node expanded	$g(n)$	$h(n)$	$f(n)$	Frontier after expansion (f values)
1	S	0	5	5	A: $g=1, h=4, f=5$ B: $g=4, h=2, f=6$
2	A	1	4	5	B: $f=6$ C: $g=3, h=2, f=5$ D: $g=6, h=1, f=7$
3	C	3	2	5	B: $f=6$ D: $f=7$ G: $g=6, h=0, f=6$
4	G (goal, min f in frontier)	6	0	6	search terminates

At step 3, A is chosen over B because it has the lower f -value ($5 < 6$). Expanding A generates C ($f = 5$), which is expanded next. C generates G with $f = 6$ — the lowest value in the frontier — so G is selected and the search stops.

Optimal path: $S \rightarrow A \rightarrow C \rightarrow G$, with total cost $g(G) = 1 + 2 + 3 = 6$.

This is cheaper than the path Greedy Best-First Search found in Q1 (cost 7), illustrating that A* — using a consistent heuristic — guarantees the optimal path, while Greedy Best-First Search does not.

Q3. CSP — 4-Queens Problem (Backtracking Search)

Formulation

Variables: Q_1, Q_2, Q_3, Q_4 — one per row of a 4×4 board; the value assigned to Q_i is the column of the queen placed in row i .

Domain (each variable): $\{1, 2, 3, 4\}$.

Constraints: for any two variables Q_i, Q_j ($i \neq j$): $Q_i \neq Q_j$ (no shared column), and $|Q_i - Q_j| \neq |i - j|$ (no shared diagonal). (Rows are automatically distinct.)

Dry run (variables assigned in order Q_1, Q_2, Q_3, Q_4 ; values tried in order 1, 2, 3, 4)

Step	Assignment tried	Constraint check	Result
1	$Q_1 = 1$	no earlier variables	Consistent — accepted
2	$Q_2 = 1$	same column as Q_1	Fail
3	$Q_2 = 2$	$ 2-1 =1= 2-1 $ — diagonal with Q_1	Fail

Step	Assignment tried	Constraint check	Result
4	$Q2 = 3$	col ok; $ 3-1 =2 \neq 1$ — no diagonal	Consistent — accepted
5	$Q3 = 1$	same column as Q1	Fail
6	$Q3 = 2$	col ok vs Q1,Q2; $ 2-3 =1= 3-2 $ — diagonal with Q2	Fail
7	$Q3 = 3$	same column as Q2	Fail
8	$Q3 = 4$	col ok; $ 4-3 =1= 3-2 $ — diagonal with Q2	Fail — domain exhausted
9	Backtrack to Q2	$Q2 = 3$ has no consistent value for Q3 → undo $Q2 = 3$, try next value	—
10	$Q2 = 4$	col ok; $ 4-1 =3 \neq 1$ — no diagonal with Q1	Consistent — accepted
11	$Q3 = 1$	same column as Q1	Fail
12	$Q3 = 2$	col ok vs Q1,Q2; $ 2-1 =1= 3-1 $? no (row diff 2) — no diagonal	Consistent — accepted
13	$Q4 = 1$	same column as Q1	Fail
14	$Q4 = 2$	same column as Q3	Fail
15	$Q4 = 3$	col ok vs Q1,Q2; $ 3-2 =1= 4-3 $ — diagonal with Q3	Fail
16	$Q4 = 4$	same column as Q2	Fail — domain exhausted
17	Backtrack to Q3	$Q3 = 2$ has no consistent value for Q4 → domain of Q3 also exhausted (1,2,3,4 all tried)	—
18	Backtrack to Q2	$Q2 = 4$ exhausted → backtrack further to Q1	—
19	$Q1 = 2$	restart search from Q1 with next value	Consistent — accepted

Continuing the same procedure from $Q1 = 2$ (omitted here for brevity, following the identical assign–check–backtrack pattern above) leads to a fully consistent assignment:

Q1	Q2	Q3	Q4
2	4	1	3

Verification: columns $\{2,4,1,3\}$ are all different. Diagonals — $|Q1-Q2|=2$ (rows differ 1, ok), $|Q1-Q3|=1$ (rows differ 2, ok), $|Q1-Q4|=1$ (rows differ 3, ok), $|Q2-Q3|=3$ (rows differ 1, ok), $|Q2-Q4|=1$ (rows differ 2, ok), $|Q3-Q4|=2$ (rows differ 1, ok). No constraint is violated.

Final solution: $Q1 = 2$, $Q2 = 4$, $Q3 = 1$, $Q4 = 3$ — queens placed at (row 1, col 2), (row 2, col 4), (row 3, col 1), (row 4, col 3).

Q4. Minimax Algorithm

Game tree (MAX at the root, alternating MIN/MAX by level, 3 levels deep, 8 terminal nodes):

Level	Player	Nodes
0 (root)	MAX	Root
1	MIN	B, C
2	MAX	D, E (children of B); F, G (children of C)
3 (leaves)	—	D → {H=3, I=5} E → {J=6, K=9} F → {L=1, M=2} G → {N=0, O=-1}

Dry run — backed-up values

Node	Type	Computation	Backed-up value
D	MAX	$\max(H=3, I=5)$	5
E	MAX	$\max(J=6, K=9)$	9
B	MIN	$\min(D=5, E=9)$	5
F	MAX	$\max(L=1, M=2)$	2
G	MAX	$\max(N=0, O=-1)$	0
C	MIN	$\min(F=2, G=0)$	0
Root	MAX	$\max(B=5, C=0)$	5

Best move for MAX: move toward B (value 5), since $\max(B=5, C=0) = 5$ is the higher of the two options at the root.

Justification: MAX assumes MIN will play optimally. Under B, MIN's best reply still yields 5 ($\min(5, 9)=5$); under C, MIN's best reply yields only 0 ($\min(2, 0)=0$). MAX therefore prefers the subtree rooted at B.

Q5. Minimax with Alpha-Beta Pruning

Same game tree as Q4. Traversal is left to right: Root → B → D → (H, I) → E → (J, K) → C → F → (L, M) → G → (N, O).

Dry run — α , β values and pruning

Node visited	α (on entry)	β (on entry)	Action	α/β after	Pruning
Root (MAX)	$-\infty$	$+\infty$	enters B	—	—
B (MIN)	$-\infty$	$+\infty$	enters D	—	—
D (MAX)	$-\infty$	$+\infty$	H=3 → v=3; I=5 → v=5	$\alpha=5$	none; D returns 5

Node visited	α (on entry)	β (on entry)	Action	α/β after	Pruning
B (MIN)	$-\infty$	$+\infty$	$v=\min(\infty,5)=5$	$\beta=5$	$\alpha(-\infty) < \beta(5)$: continue to E
E (MAX)	$-\infty$	5	$J=6 \rightarrow v=6,$ $\alpha=\max(-\infty,6)=6$	$\alpha=6$	$\alpha(6) \geq \beta(5)$: PRUNE K; E returns 6
B (MIN)	$-\infty$	5	$v=\min(5,6)=5$	returns 5	B's final value = 5
Root (MAX)	$-\infty$	$+\infty$	$v=\max(-\infty,5)=5$	$\alpha=5$	$\alpha(5) < \beta(+\infty)$: continue to C
C (MIN)	5	$+\infty$	enters F	—	—
F (MAX)	5	$+\infty$	$L=1 \rightarrow v=1; M=2 \rightarrow$ $v=2$	returns 2	no prune inside F
C (MIN)	5	$+\infty$	$v=\min(\infty,2)=2$	$\beta=2$	$\alpha(5) \geq \beta(2)$: PRUNE G (and N, O); C returns 2
Root (MAX)	5	$+\infty$	$v=\max(5,2)=5$	returns 5	Root's final value = 5

Pruned branches: node K (under E) is cut off because $\alpha(6) \geq \beta(5)$ at E — once MIN (at B) already has an alternative worth 5, MAX exploring E is guaranteed at least 6, so B will never choose E's other children; the exact value of K cannot lower MIN's choice and is irrelevant.

Node G, together with its children N and O (under C), is cut off because $\alpha(5) \geq \beta(2)$ at C — MAX already has an option (B) worth 5, so as soon as C is shown to be worth only 2, MAX will never choose C regardless of G's value.

Final best move: identical to Q4 — MAX moves toward B, with backed-up value 5.

Alpha-beta pruning reaches this same result while evaluating fewer leaf nodes (K, N, and O were never examined).